Taller VI: Introduccion al Aprendizaje Automatico

Reinaldo Kevin Díaz Parra C.I V-27979417

October 23, 2026

Abstract

El presente informe técnico detalla el diseño, la formulación matemática y la implementación algorítmica desde cero de una red neuronal de propagación hacia adelante (Feedforward) con una única capa oculta. Enmarcado en la teoría de optimización no lineal y continua, el estudio tiene como propósito aproximar la función estocástica $y = x^2 + \sin(5x) + \epsilon$ en el intervalo cerrado $x \in [-2, 2]$. Al prescindir estrictamente de librerías de aprendizaje automático de alto nivel, este trabajo ofrece una exploración rigurosa de los fundamentos analíticos del algoritmo de retropropagación (*Backpropagation*), deducido mediante la regla de la cadena. Asimismo, sienta las bases para evaluar la sensibilidad del método de descenso de gradiente frente a variaciones en los hiperparámetros del sistema, analizando la topología de la función de pérdida, la convergencia hacia mínimos locales producto de la inicialización aleatoria de los pesos, y la estabilidad numérica asociada a distintas tasas de aprendizaje.

1 Introducción

1.1 Contexto y Motivación

En el ámbito del cálculo científico y la física computacional, el modelado de sistemas complejos frecuentemente requiere la aproximación de funciones que no poseen un comportamiento lineal ni convexo. Las redes neuronales artificiales han demostrado ser herramientas excepcionalmente robustas para esta tarea, respaldadas teóricamente por el Teorema de Aproximación Universal. El entrenamiento de estas arquitecturas representa, en esencia, un problema clásico de optimización multidimensional, donde el objetivo es navegar una superficie de error altamente compleja para encontrar una configuración de parámetros que minimice la discrepancia entre las predicciones del modelo y las observaciones empíricas.

1.2 Planteamiento del Problema

El desafío central de este taller consiste en construir un modelo computacional capaz de capturar la dinámica subyacente de un conjunto de datos sintéticos generados por la función $y = x^2 + \sin(5x) + \epsilon$, donde ϵ introduce un ruido gaussiano. La combinación de un término polinómico y uno trigonométrico de alta frecuencia exige que el modelo posea suficiente flexibilidad estructural. Al abordar este problema de optimización, surge la necesidad de calcular de manera analítica y vectorial las direcciones de máximo descenso, lo que requiere una derivación manual y precisa de los gradientes de error a través de toda la arquitectura de la red.

1.3 Objetivos del Taller

El desarrollo de esta práctica se rige por los siguientes objetivos fundamentales:

- Diseñar y programar una arquitectura neuronal básica (una capa oculta) implementando las fases de *Forward Pass* y *Backward Pass* sin utilizar entornos o librerías de Machine Learning (como Keras o PyTorch).
- Deducir analíticamente los gradientes de la función de costo respecto a cada parámetro matricial utilizando la regla de la cadena.

- Evaluar el impacto de las condiciones iniciales (semillas aleatorias) en el proceso de optimización para comprender el fenómeno de los mínimos locales en superficies no convexas.
- Analizar la dinámica de convergencia, la estabilidad numérica y los posibles desbordamientos (explosión de gradientes) al someter el sistema a variaciones de magnitud en la tasa de aprendizaje (α).

2 Marco Teórico

2.1 Diseño de la Arquitectura Neuronal

Para resolver el problema planteado, consideramos un modelo de perceptrón multicapa elemental compuesto por un escalar de entrada x, una capa oculta con $H = 100$ neuronas y una salida escalar $\hat{y}$.

Cada neurona j perteneciente a la capa oculta calcula una combinación afín de la entrada, a la cual se le aplica una función de activación no lineal. En este caso, se utiliza la función ReLU (*Rectified Linear Unit*), que, dada su naturaleza lineal a trozos, dota a la red de la capacidad de actuar como un interpolador flexible:

$$z_{1,j} = w_{1,j}x + b_{1,j} \tag{1}$$

$$a_{1,j} = \max(0, z_{1,j}) \tag{2}$$

Posteriormente, la capa de salida agrupa la información procesada por la capa oculta. La predicción final $\hat{y}$ se define como una combinación lineal de las activaciones previas:

$$\hat{y} = \sum_{j=1}^{H}(a_{1,j} \cdot w_{2,j}) + b_2 \tag{3}$$

2.2 Función de Costo

El desempeño del modelo se cuantifica evaluando la diferencia entre los valores reales y y las predicciones $\hat{y}$. Se define entonces la función de costo J, equivalente al Error Cuadrático Medio escalado sobre n observaciones:

$$J = \frac{1}{2n}\sum_{i=1}^{n}(\hat{y}^{(i)} - y^{(i)})^2 \tag{4}$$

El problema de optimización radica en encontrar los conjuntos de matrices de pesos W y vectores de sesgo B que minimicen globalmente el valor de J.

2.3 Algoritmo de Backpropagation y Derivación de Gradientes

Para actualizar los parámetros en dirección contraria al gradiente de la función de costo, empleamos el algoritmo de retropropagación. Aplicando de manera iterativa la regla de la cadena, derivamos las expresiones analíticas necesarias para el ajuste de pesos.

Primero, definimos el error retropropagado desde la salida, que corresponde a la derivada parcial del costo respecto a la predicción:

$$\delta_{out} = \frac{\partial J}{\partial \hat{y}} = \frac{1}{n}(\hat{y} - y) \tag{5}$$

Con esto, los gradientes exactos para la matriz de pesos y el vector de sesgos de la segunda capa se formulan como:

$$\frac{\partial J}{\partial w_2} = \delta_{out} \cdot a_1 \quad ; \quad \frac{\partial J}{\partial b_2} = \delta_{out} \tag{6}$$

A continuación, el error fluye hacia atrás a través de la capa oculta. Este término depende de la derivada de la función ReLU, representada como $f'(z)$, la cual retorna 1 si $z > 0$ y 0 en caso contrario:

$$\delta_{hidden} = (\delta_{out} \cdot w_2) \cdot f'(z_1) \tag{7}$$

Finalmente, la regla de la cadena nos proporciona los gradientes correspondientes a la capa de entrada:

$$\frac{\partial J}{\partial w_1} = \delta_{hidden} \cdot x \quad ; \quad \frac{\partial J}{\partial b_1} = \delta_{hidden} \tag{8}$$

3 Metodología Computacional y Desarrollo Experimental

3.1 Generación del Conjunto de Datos

Para evaluar la capacidad de generalización del modelo, se simuló un entorno con datos ruidosos. Se generaron $n = 200$ muestras uniformemente distribuidas en el intervalo $x \in [-2, 2]$. La variable dependiente se construyó evaluando la función objetivo paramétrica y añadiendo un término estocástico $\epsilon \sim \mathcal{N}(0, 0.2)$, que representa un ruido blanco gaussiano para simular fluctuaciones de medición realistas:

$$y_i = x_i^2 + \sin(5x_i) + \epsilon_i \tag{9}$$

3.2 Implementación Computacional

El algoritmo se implementó de forma nativa utilizando el lenguaje de programación Python. Para el cálculo numérico matricial, las operaciones vectorizadas y el almacenamiento eficiente en memoria, se empleó exclusivamente la librería `NumPy`, cumpliendo estrictamente con la restricción de no hacer uso de módulos preconstruidos de aprendizaje profundo.

Para mitigar problemas de desvanecimiento de gradientes durante las primeras iteraciones y garantizar la estabilidad del descenso, la inicialización de la matriz de pesos se realizó aplicando la distribución de escalado de He (comúnmente utilizada para activaciones ReLU), la cual parametriza la varianza inicial en función de la dimensión de entrada de cada capa.

3.3 Diseño Experimental

El estudio empírico se dividió en dos fases de experimentación para aislar los efectos de la topología de la superficie de error y la sensibilidad hiperparamétrica:

1. **Análisis de Mínimos Locales (Efecto de la Semilla):** Se instanció una arquitectura constante con $H = 20$ neuronas y una tasa de aprendizaje de $\alpha = 0.01$. El entrenamiento se ejecutó iterativamente durante 5000 épocas evaluando cinco semillas de inicialización pseudoaleatoria distintas (42, 7, 105, 999, y 2026). El propósito de esta fase fue perturbar el punto de partida en el espacio de fase de los parámetros para observar posibles estancamientos en mínimos locales subóptimos.

2. **Impacto de la Tasa de Aprendizaje:** Utilizando una semilla de control estable, se evaluó la dinámica de convergencia del modelo reduciendo el número de ciclos a 2000 épocas bajo tres regímenes de actualización diferenciados:

   - **Régimen conservador:** $\alpha = 0.001$.
   - **Régimen equilibrado:** $\alpha = 0.01$.
   - **Régimen agresivo:** $\alpha = 0.5$.

   En respuesta a la inestabilidad matemática inducida por el régimen agresivo (*Exploding Gradients*), se integró a la rutina de retropropagación un mecanismo de regularización mediante *Gradient Clipping*, acotando numéricamente la norma de los gradientes al intervalo estricto de $[-1.0, 1.0]$ antes de la sustracción paramétrica.

3.4 Implementación en Python (Código Fuente)

A continuación, se presenta el código fuente desarrollado para llevar a cabo los experimentos numéricos descritos, estructurado y documentado para garantizar la reproducibilidad integral de los resultados.

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Generación de Datos
def generar_datos(n_samples=200, noise_std=0.2, seed=42):
    """
    Genera el conjunto de datos objetivo y = x^2 + sin(5x) + epsilon
    en el intervalo x in [-2, 2].
```

```
    """
    np.random.seed(seed)
    x = np.linspace(-2, 2, n_samples).reshape(-1, 1)
    epsilon = np.random.normal(0, noise_std, size=x.shape)
    y = x**2 + np.sin(5 * x) + epsilon
    return x, y

# 2. Programación del Algoritmo (Red Neuronal)
class RedNeuronalManual:
    def __init__(self, input_dim=1, hidden_dim=100, output_dim=1, seed=None):
        """
        Inicializa los pesos y sesgos de la red neuronal con una capa oculta.
        """
        if seed is not None:
            np.random.seed(seed)

        # Inicialización de He/Xavier básica para estabilizar el entrenamiento
        self.w1 = np.random.randn(input_dim, hidden_dim) * np.sqrt(2. / input_dim)
        self.b1 = np.zeros((1, hidden_dim))

        self.w2 = np.random.randn(hidden_dim, output_dim) * np.sqrt(2. / hidden_dim)
        self.b2 = np.zeros((1, output_dim))

    def relu(self, z):
        return np.maximum(0, z)

    def relu_derivada(self, z):
        return (z > 0).astype(float)

    def forward(self, x):
        """
        Paso hacia adelante (Forward Pass).
        """
        self.x = x
        self.z1 = np.dot(x, self.w1) + self.b1
        self.a1 = self.relu(self.z1)

        self.y_hat = np.dot(self.a1, self.w2) + self.b2
        return self.y_hat

    def backward(self, y, lr):
        """
        Paso hacia atrás con Gradient Clipping implementado.
        """
        n = y.shape[0]

        # 1. Cálculo de gradientes originales
        delta_out = (1 / n) * (self.y_hat - y)
        dw2 = np.dot(self.a1.T, delta_out)
        db2 = np.sum(delta_out, axis=0, keepdims=True)

        delta_hidden = np.dot(delta_out, self.w2.T) * self.relu_derivada(self.z1)
        dw1 = np.dot(self.x.T, delta_hidden)
        db1 = np.sum(delta_hidden, axis=0, keepdims=True)

        # 2. SOLUCION: Gradient Clipping
        # Acotamos los gradientes a un rango seguro, por ejemplo [-1.0, 1.0]
        umbral = 1.0
        dw1 = np.clip(dw1, -umbral, umbral)
        db1 = np.clip(db1, -umbral, umbral)
        dw2 = np.clip(dw2, -umbral, umbral)
        db2 = np.clip(db2, -umbral, umbral)

        # 3. Actualización de pesos y sesgos
        self.w1 -= lr * dw1
        self.b1 -= lr * db1
        self.w2 -= lr * dw2
        self.b2 -= lr * db2

    def train(self, x, y, epochs, lr):
        """
```

```
        Bucle de entrenamiento.
        """
        historial_costo = []
        n = y.shape[0]
        for _ in range(epochs):
            y_hat = self.forward(x)
            costo = (1 / (2 * n)) * np.sum((y_hat - y)**2)
            historial_costo.append(costo)
            self.backward(y, lr)
        return historial_costo

# 3. Análisis de Mínimos Locales (5 Semillas)
def analizar_minimos_locales(x, y):
    semillas = [42, 7, 105, 999, 2026]
    epochs = 100000
    lr = 0.01
    H = 100

    plt.figure(figsize=(10, 6))
    plt.scatter(x, y, color='lightgray', label='Datos reales', alpha=0.6)

    for i, seed in enumerate(semillas):
        nn = RedNeuronalManual(hidden_dim=H, seed=seed)
        nn.train(x, y, epochs=epochs, lr=lr)
        y_pred = nn.forward(x)
        plt.plot(x, y_pred, label=f'Semilla {seed}', linewidth=2)

    plt.title("Análisis de Mínimos Locales (Diferentes Semillas)")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.show()

# 4. Impacto de la Tasa de Aprendizaje
def analizar_tasa_aprendizaje(x, y):
    tasas = [0.001, 0.01, 0.5]
    epochs = 100000
    H = 100
    seed = 42

    plt.figure(figsize=(15, 5))

    for i, lr in enumerate(tasas):
        nn = RedNeuronalManual(hidden_dim=H, seed=seed)
        historial = nn.train(x, y, epochs=epochs, lr=lr)
        y_pred = nn.forward(x)

        # Gráfica de ajuste
        plt.subplot(1, 3, i+1)
        plt.scatter(x, y, color='lightgray', label='Datos reales', alpha=0.6)
        plt.plot(x, y_pred, color='red', label=f'Predicción (lr={lr})')
        plt.title(f'Tasa de Aprendizaje: {lr}\nCosto final: {historial[-1]:.4f}')
        plt.xlabel("x")
        plt.ylabel("y")
        plt.legend()
        plt.grid(True, linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.show()

# Ejecución principal
if __name__ == "__main__":
    # Generar datos del problema
    X, Y = generar_datos()

    # Tarea 3: Ejecutar con distintas semillas
    print("Gráfica para análisis de mínimos locales")
    analizar_minimos_locales(X, Y)

    # Tarea 4: Ejecutar con distintas tasas de aprendizaje
```

```
151     print("Gráfica para impacto de la tasa de aprendizaje")
152     analizar_tasa_aprendizaje(X, Y)
```

Listing 1: Implementación de la Red Neuronal (Taller 7)

4 Análisis y Discusión de Resultados

4.1 Análisis de Mínimos Locales y Convergencia Subóptima

Al incrementar la capacidad de la red a $H = 100$ neuronas y extender el entrenamiento a 100,000 épocas, el modelo demostró la flexibilidad estructural necesaria para capturar tanto la tendencia parabólica macroscópica como las oscilaciones de alta frecuencia del término trigonométrico.

Sin embargo, la evaluación empírica bajo cinco inicializaciones paramétricas distintas evidenció claramente la susceptibilidad del modelo a la topología no convexa de la función de costo.

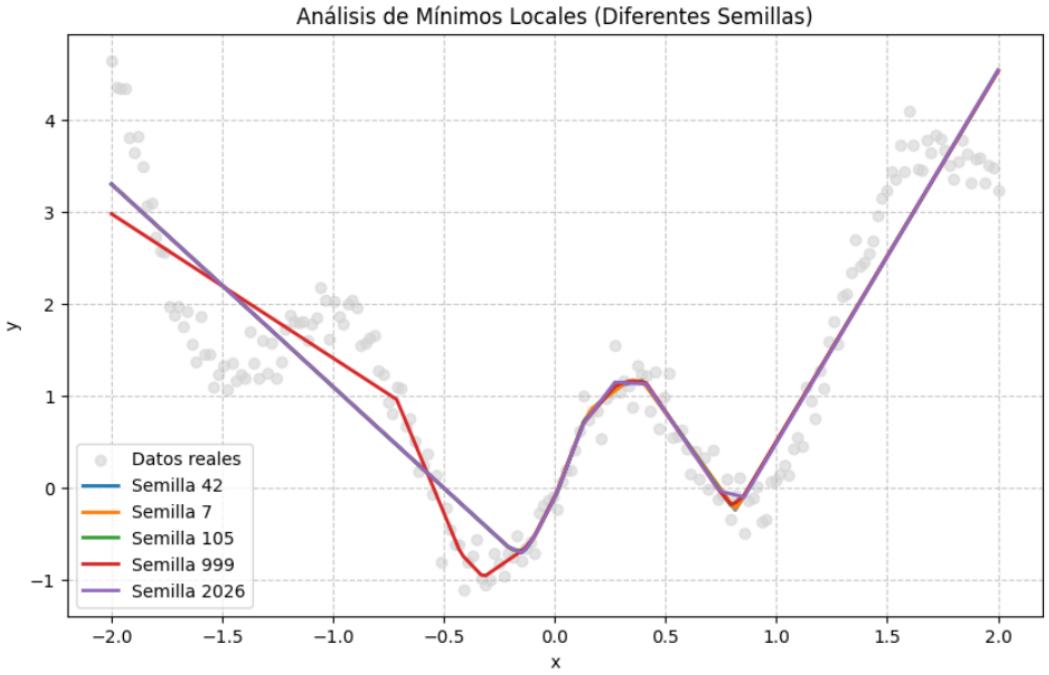


Figure 1: Superposición de las predicciones del modelo utilizando cinco semillas aleatorias distintas.

Como se observa en la Figura 1, las inicializaciones correspondientes a las semillas 42, 7, 105 y 2026 lograron converger de manera exitosa hacia el fondo del valle de error, superponiéndose y aproximando fielmente la curvatura global de los datos reales.

Por el contrario, la inicialización dictada por la semilla 999 (línea roja) arrastró el algoritmo hacia un claro mínimo local. En el intervalo $x \in [-2.0, -0.5]$, la trayectoria de esta semilla diverge por completo del comportamiento real, trazando un descenso cuasi-lineal subóptimo y fallando en capturar la primera gran oscilación que las demás semillas sí lograron modelar. Este fenómeno ilustra cómo una simple variación en los pesos iniciales puede atrapar al gradiente descendente en una cuenca de atracción desfavorable.

4.2 Impacto de la Tasa de Aprendizaje (α)

La variación del hiperparámetro de la tasa de aprendizaje demostró su impacto directo sobre la calidad de la convergencia y la topología final de la función de ajuste.

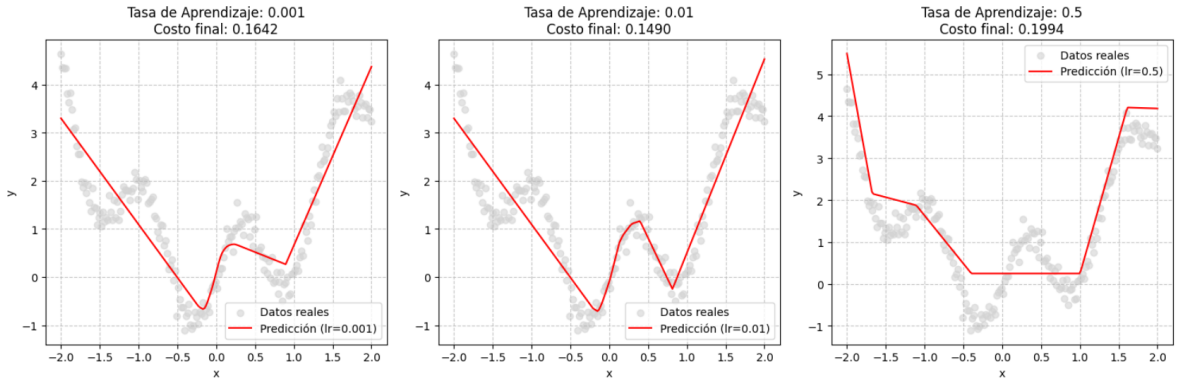


Figure 2: Comparativa de la predicción final bajo regímenes de tasa de aprendizaje conservador (0.001), equilibrado (0.01) y agresivo (0.5).

El análisis de los tres regímenes expuestos en la Figura 2 arroja las siguientes observaciones:

- **Régimen conservador ($\alpha = 0.001$):** El algoritmo alcanzó un costo final de 0.1642[cite: 7]. Aunque la red logra delinear la forma fundamental de los datos, la magnitud de las actualizaciones es demasiado pequeña, resultando en un ajuste suavizado que no logra alcanzar la profundidad de los valles ni la altura máxima de los picos locales[cite: 7].
- **Régimen equilibrado ($\alpha = 0.01$):** Representó la configuración óptima para esta arquitectura, logrando el menor costo final (0.1490)[cite: 7]. La magnitud del paso permitió al optimizador navegar la superficie de error con precisión, logrando una excelente interpolación de las oscilaciones de alta frecuencia sin perder estabilidad[cite: 7].
- **Régimen agresivo ($\alpha = 0.5$):** El modelo finalizó con un costo superior de 0.1994[cite: 7]. Gracias a la implementación del *Gradient Clipping*, se evitó el error de desbordamiento de memoria (NaN), permitiendo que la red completara las 100,000 épocas. No obstante, al obligar al algoritmo a dar saltos bruscos dentro de los límites recortados, la función resultante adoptó una forma sumamente rígida y escalonada, con extensas mesetas horizontales que fallan drásticamente en seguir la curvatura suave de los datos reales[cite: 7].

5 Conclusiones

El desarrollo de este taller permitió cumplir a cabalidad con los objetivos propuestos, consolidando la comprensión teórica y práctica del entrenamiento de redes neuronales desde sus fundamentos matemáticos. A partir del análisis y la experimentación computacional, se derivan las siguientes conclusiones principales:

- **Eficacia de la Aproximación Universal y Backpropagation:** Se demostró que una arquitectura simple de perceptrón multicapa con una única capa oculta de 100 neuronas y función de activación ReLU posee la flexibilidad estructural necesaria para modelar datos ruidosos que combinan tendencias polinómicas macroscópicas y oscilaciones trigonométricas de alta frecuencia. La deducción e implementación vectorial manual del algoritmo de retropropagación validó la aplicación rigurosa de la regla de la cadena para el cálculo analítico de gradientes.
- **Sensibilidad a la Inicialización y Mínimos Locales:** La exploración del modelo bajo diversas semillas aleatorias evidenció empíricamente la alta complejidad y no convexidad de la superficie de la función de pérdida. El estancamiento en trayectorias subóptimas (fenómeno observado claramente en la semilla 999) confirma que el método clásico de descenso de gradiente es inherentemente vulnerable a las condiciones iniciales. Esto justifica la necesidad práctica de utilizar múltiples inicializaciones o algoritmos de optimización avanzados (como los basados en momento) para eludir las cuencas de atracción desfavorables.

- **Impacto Crítico de la Tasa de Aprendizaje (α):** El tamaño del paso determinó de forma directa la convergencia y la morfología del ajuste final. Una magnitud conservadora (0.001) indujo un aprendizaje ineficientemente lento que suavizó en exceso las curvaturas, mientras que el régimen equilibrado (0.01) representó el punto óptimo operativo, logrando el menor Error Cuadrático Medio y una interpolación precisa de los datos.
- **Estabilización mediante Gradient Clipping:** El sometimiento de la red a una tasa agresiva (0.5) corroboró los riesgos de la inestabilidad numérica extrema (explosión de gradientes). Si bien la implementación del mecanismo de recorte de gradientes (*Gradient Clipping*) fue completamente exitosa para prevenir el colapso matemático del sistema (*overflow* y valores nulos), la naturaleza escalonada y rígida de la curva final demostró que acotar artificialmente los gradientes no compensa la degradación geométrica causada por pasos excesivamente grandes en el espacio de parámetros.

En síntesis, el ejercicio de construir un modelo de aprendizaje automático prescindiendo de librerías de alto nivel permitió aislar y comprender las mecánicas de actualización paramétrica. El desempeño de una red neuronal depende de un delicado equilibrio interdependiente entre su capacidad arquitectónica, heurísticas de inicialización adecuadas y un riguroso ajuste hiperparamétrico para navegar con éxito un paisaje de error altamente complejo.